

CASH MAKER

Documento de Requisitos do Produto

PRD · v1.0 · 2026

Sistema web de gestão financeira pessoal · Stack Next.js · PostgreSQL

MVP Definido

33 Requisitos

4 Fases

01

Visão Geral

O Cash Maker é uma plataforma web de gestão financeira pessoal, projetada para oferecer controle total sobre receitas, despesas, metas e padrões de consumo. Nasceu como projeto pessoal e evolui para um produto comercial escalável.

Proposta de valor

Clareza sem complexidade. O Cash Maker entrega visão real das finanças com uma interface premium — leve como vidro, sólida como dados. Lançar uma transação em menos de 10 segundos. Dashboard com visão 360° na primeira tela.

1.1 Objetivo do Sistema

Fornecer uma plataforma simples, automatizada e visual para controle financeiro pessoal — reduzindo o esforço manual e aumentando a clareza sobre gastos e padrões financeiros.

1.2 Público-Alvo

Perfil	Descrição
Usuário principal (MVP)	Pessoa física que quer organizar finanças pessoais de forma simples
Usuário intermediário	Quem quer automatizar controle via importação de extratos bancários
Usuário avançado (futuro)	Quem busca relatórios detalhados, alertas automáticos e insights de comportamento financeiro

1.3 Stack Tecnológica

Camada	Tecnologia	Justificativa
Frontend	Next.js 15+ (App Router)	SSR, roteamento, API Routes integradas
Estilo	Tailwind CSS v4 + shadcn/ui	Componentes prontos e customizáveis, velocidade de desenvolvimento, tema verde aplicado via design tokens
Gráficos	Recharts	Dashboards financeiros com animações suaves
Backend	Next.js API Routes (Node.js)	Unificado, sem servidor separado no MVP
Banco de Dados	PostgreSQL + Prisma ORM	ACID, escalável, type-safe — ideal para dados financeiros

DB Hosting	Neon.tech (free tier)	PostgreSQL serverless, gratuito para MVP
Deploy	Vercel (free tier)	Zero config para Next.js, CI/CD automático

02

Requisitos Funcionais

Todos os comportamentos que o sistema deve executar, organizados por módulo funcional.

2.1 Autenticação

RF01	Cadastro de usuário	Nome, e-mail e senha. Validação de e-mail único e força da senha.
RF02	Login com e-mail e senha	Autenticação via JWT/cookies seguros. Feedback de erro claro.
RF03	Logout	Encerramento de sessão com limpeza de token e redirecionamento.
RF04	Recuperação de senha	Fluxo via e-mail com link temporário. Planejado para v1.1.
RF05	Persistência de sessão	Sessão mantida entre abas e recarregamentos via cookie seguro.

2.2 Transações

RF06	Criar transação	Campos: tipo (receita/despesa), valor, data, categoria, descrição. Fluxo em < 10s.
RF07	Editar transação	Edição inline ou via modal. Todos os campos editáveis pós-criação.
RF08	Excluir transação	Soft delete — registro permanece no banco, invisível no dashboard e relatórios.
RF09	Listar transações	Lista paginada com filtros por data, tipo e categoria. Ordenação configurável.

2.3 Categorias

RF10	Categorias padrão	Sistema inicia com: Alimentação, Transporte, Moradia, Saúde, Lazer, Educação, Outros.
RF11	Criar categorias personalizadas	Nome, ícone (Lucide) e cor. Vinculadas ao usuário, não compartilhadas.
RF12	Editar categorias	Edição de nome, ícone e cor. Transações já vinculadas mantêm a categoria atualizada.

RF13	Excluir categorias	Só se não houver transações vinculadas. Caso contrário, solicitar reatribuição.
------	--------------------	---

2.4 Gastos Fixos

RF14	Criar gasto fixo	Nome, valor, frequência (mensal/semanal/anual) e data de início.
RF15	Gerar transações automaticamente	Cron job gera transações pendentes conforme frequência definida.
RF16	Ativar / desativar gasto fixo	Toggle que suspende geração futura sem excluir o registro.
RF17	Identificar gastos fixos via extrato	Sistema detecta padrões de recorrência no extrato importado e sugere criação de gasto fixo.

02

Requisitos Funcionais — continuação

2.5 Dashboard

RF18	Exibir saldo atual	Saldo em destaque no topo — receitas menos despesas do período selecionado.
RF19	Receitas e despesas por período	Cards de resumo (dia / semana / mês / ano) com variação percentual.
RF20	Gráficos por categoria	Pizza ou donut com distribuição de gastos por categoria no período.
RF21	Evolução financeira mensal	Gráfico de barras ou linha com histórico dos últimos 6–12 meses.
RF22	Filtros interativos	Filtro por data, categoria e tipo de transação aplicados em tempo real.

2.6 Relatórios

RF23	Relatório mensal	Resumo de receitas, despesas, saldo e top categorias do mês.
RF24	Exportação CSV	Download do extrato de transações filtradas. Planejado para v1.1.

2.7 Importação de Extrato (Feature Avançada)

Por que é diferencial?

A maioria dos apps de finanças exige lançamento manual. A importação de extrato elimina esse atrito — o usuário sobe o arquivo e o sistema faz o trabalho pesado, detectando padrões e sugerindo categorias automaticamente.

RF25	Upload de extrato bancário	Suporte a OFX, CSV e PDF (conforme banco). Limite de 10MB por arquivo.
RF26	Seleção do banco de origem	Dropdown com bancos suportados (Bradesco, Itaú, Nubank, inter, C6 — fase 1).
RF27	Processamento automático	Parser interpreta o arquivo e extrai: data, valor, descrição, tipo.
RF28	Identificação de receitas e despesas	Classificação automática com base no sinal do valor e palavras-chave da descrição.

RF29	Tela de revisão	Usuário vê todos os registros extraídos antes de confirmar — pode editar cada um.
RF30	Edição antes de salvar	Por transação: definir categoria, marcar como gasto fixo, corrigir valores.
RF31	Deteção de recorrência	Sistema identifica padrões (mesmo valor + descrição em datas regulares) e sugere gasto fixo.

2.8 Notificações por E-mail

RF32	Envio de alertas por e-mail	Integração com serviço de e-mail transacional (Resend ou SendGrid).
RF33	Saldo negativo	Alerta imediato quando o saldo cai abaixo de zero.
RF34	Gastos elevados	Alerta quando gasto em categoria superar 80% do histórico médio mensal.
RF35	Contas recorrentes detectadas	Notificação quando o sistema detectar novo padrão de recorrência no extrato.

03

Requisitos Não Funcionais

Qualidades que o sistema deve ter além do comportamento funcional — segurança, performance, escalabilidade e manutenibilidade.

RNF01	Hash de senhas	bcrypt com salt rounds >= 12. Nunca armazenar senha em texto puro.
RNF02	Autenticação segura	JWT com expiração curta + refresh token, ou sessão via cookies HttpOnly/Secure.
RNF03	Isolamento de dados	Todas as queries filtradas por userId — um usuário nunca acessa dados de outro.
RNF04	Proteção contra ataques	Sanitização de inputs (XSS), CSRF tokens, validação com Zod, queries via Prisma (sem SQL bruto).
RNF05	Performance da API	Tempo de resposta < 500ms para 95% das requests em condições normais.
RNF06	Queries otimizadas	Índices no banco para campos de busca frequente (userId, date, categoryId).
RNF07	Responsividade	Layout funcional em mobile (360px+), tablet (768px+) e desktop (1024px+).
RNF08	Alta disponibilidade	Vercel + Neon garantem SLA adequado para MVP sem infraestrutura própria.
RNF09	Escalabilidade	Arquitetura stateless que permite scale horizontal sem refatoração estrutural.
RNF10	Testes	Suporte a testes unitários (Jest) e integração (Supertest). Meta: cobertura > 60% no MVP.
RNF11	Arquitetura em camadas	Separação: route handler → service → repository → banco. Sem lógica de negócio nas rotas.
RNF12	Versionamento	Git com branches por feature (feature/nome), PRs para main. Commits semânticos.
RNF13	Interface intuitiva	Componentes shadcn/ui como base. Qualquer ação principal acessível em no máximo 3 cliques a partir do dashboard.
RNF14	Feedback visual	Toasts via shadcn/ui Sonner, skeleton screens, loading states. Zero ações silenciosas.
RNF15	Transações atômicas	Operações financeiras que afetam múltiplas tabelas usam Prisma transactions.

RNF16	Localização	Fuso fixo: America/Sao_Paulo (UTC-3). Datas exibidas no padrão dd/mm/yyyy.
-------	-------------	--

04

Regras de Negócio

Lógica específica do domínio que define como o sistema deve se comportar em situações particulares.

RN01**Saldo negativo**

- Saldo negativo é permitido no sistema
- Exibir alerta visual em vermelho no dashboard
- Disparar notificação por e-mail ao usuário
- Badge de alerta persistente até saldo ser regularizado

RN02**Gastos fixos**

- Cron job executa diariamente às 00:01 (Brasília)
- Gera transações com status 'auto' para identificação
- Gasto fixo inativo não gera novas transações
- Editar valor do gasto fixo não altera transações já geradas

RN03**Importação de extrato**

- Arquivo é processado em background (não bloqueia UI)
- Duplicatas identificadas por: data + valor + descrição similar
- Usuário deve confirmar antes de salvar qualquer transação importada
- Importação com erro parcial salva apenas os registros válidos

RN04**Soft delete de transações**

- Transação deletada recebe flag deleted_at com timestamp
- Não aparece em dashboard, relatórios ou saldo
- Visível apenas na tela de histórico, marcada em vermelho
- Não pode ser deletada permanentemente pela interface (apenas admin)

RN05**Categorias**

- 7 categorias padrão criadas automaticamente no cadastro do usuário
- Categorias padrão não podem ser excluídas, apenas renomeadas
- Categoria com transações vinculadas não pode ser excluída

- Transação sem categoria recebe 'Outros' como padrão

RN06**Datas e fusos**

- Todas as datas salvas no banco em UTC
- Conversão para America/Sao_Paulo apenas na exibição
- Relatórios mensais consideram 00:00 até 23:59 de Brasília
- Data de importação de extrato segue a data do lançamento original

05

MVP & Roadmap de Produto

Estratégia de lançamento faseada — do núcleo funcional até o produto comercial completo.

5.1 Escopo do MVP

Princípio do MVP

Entregar o máximo de valor com o mínimo de complexidade. O MVP deve ser utilizável no dia 1 — não perfeito, mas funcional e confiável.

INCLUSO no MVP	FORA do MVP (futuro)
✓ Login e cadastro de usuário	■ Importação de extrato bancário
✓ CRUD completo de transações	■ Gastos fixos automáticos (cron)
✓ Categorias (padrão + personalizadas)	■ Alertas por e-mail
✓ Saldo em tempo real	■ Exportação CSV
✓ Dashboard com cards e gráficos básicos	■ Gráficos avançados e relatórios
✓ Filtros por período e categoria	■ Detecção de recorrência por IA

5.2 Roadmap de Fases

MVP	Núcleo funcional	Auth · CRUD transações · Categorias · Dashboard básico · Deploy Vercel
v1.0	Produto completo	Gastos fixos + cron · Alertas por e-mail · Gráficos avançados · Exportação CSV · Dark mode
v1.1	Feature diferencial	Importação de extrato · Detecção de recorrência · Tela de revisão · Bancos suportados
v2.0	Produto comercial	Multi-usuário / times · Planos pagos · API pública · Integração bancária direta · Mobile app

Proximos Passos

Sequência ideal para iniciar o desenvolvimento — do banco de dados ao frontend. Cada etapa é pré-requisito da seguinte.

01	Modelagem do Banco (Prisma Schema) Definir todas as entidades: User, Transaction, Category, FixedExpense, ImportedStatement. Relacionamentos, índices e tipos. Base de tudo.
02	Definição da API (Endpoints) Mapear todas as rotas REST: POST /transactions, GET /dashboard, PATCH /categories/:id etc. Definir contratos de request/response antes de codar.
03	Estrutura do Backend Criar a arquitetura em camadas no Next.js: route handlers → services → repositories. Configurar Prisma, autenticação e middlewares.
04	Construção do Frontend Implementar as telas com Tailwind CSS + shadcn/ui seguindo o Design System Liquid Glass: layout, componentes customizados com tema verde, integração com API e estados de loading/erro.
05	Testes e Deploy Testes unitários dos services, integração das rotas críticas. Deploy no Vercel com variáveis de ambiente e banco Neon configurado.